Support Ticket Priority Classifier — 13-Minute Presentation Script

3 speakers · ~13:00 total · mapped to the 13 slides in presentation.html

Speaker key: **A** = Andrea Alarcón (Data & Backend) · **D** = Dalton Kern (Frontend & Presentation Lead) · **JJ** = Juan José Rincón (ML Engineer)

Slide 1 — Title (D · ~0:25)

"Good morning/afternoon, everyone. We're Andrea Alarcón, Dalton Kern, and Juan José Rincón, and today we're presenting our Deep Learning final project: a Support Ticket Priority Classifier. It's a Bidirectional LSTM model that automatically reads a customer support ticket and predicts how urgent it is — Low, Medium, or High — trained on 16,338 English-language support tickets."

Slide 2 — The Problem (D · ~1:05)

"Let's start with the problem we're solving. Right now, support teams triage tickets manually — an agent reads each one and decides how urgent it is before routing it. That breaks down at scale in five ways: it's slow and repetitive, it's inconsistent — two agents rate the same ticket differently — critical tickets get buried behind routine ones, round-the-clock coverage needs round-the-clock staffing, and the problem compounds as volume grows.

On average, manual triage costs about 3 minutes per ticket. At 300 tickets a day, that's 900 minutes — 15 hours — spent every single day just deciding what's urgent, not solving anything. Annualized, that's roughly 2.5 full-time salaries spent on triage alone — work that delivers zero value to the customer."

Slide 3 — Business Value (D · ~0:55)

"So what's the business case for automating this? Three things. First, time savings: triage drops from 3 minutes to under a second per ticket, freeing agents to focus on resolution, and it scales to any volume without adding headcount. Second, revenue protection: critical tickets get escalated in real time instead of hours later, reducing churn and improving SLA compliance — we estimate around a 15% lift in CSAT. Third, operational efficiency: classification is consistent 24/7, department routing is automatic, and a confidence score flags borderline cases for human review instead of guessing."

Transition: "Andrea will walk you through what we actually built."

Slide 4 — Our Solution (A · ~1:05)

"Thanks, Dalton. So what did we build? Two pieces. On the backend, a Bidirectional LSTM trained on our 16,338-ticket dataset outputs one of three priority classes plus a confidence score per class, served through a FastAPI `/predict` endpoint. On the frontend, a Streamlit app gives agents a familiar email-style interface — just a Subject and a Body field — and returns a prediction in under a second, plus a department routing recommendation and a session history. You can see a mockup here: an agent pastes in a ticket, hits classify, and gets back a priority — in this case High — routed straight to Billing, with the full confidence breakdown underneath."

Slide 5 — Dataset & EDA (A · ~1:05)

"Before training anything, we needed to understand our data. We're using a Kaggle multilingual customer support ticket dataset — 28,587 tickets across English and German — which we filtered down to 16,338 English-only tickets across three priority classes. We split it 70/15/15 into train, validation, and test, stratified by priority with a fixed random seed for reproducibility — that's 11,436 training tickets, 2,451 each for validation and test. Two EDA findings shaped how we trained: the classes turned out to be imbalanced — Low is only 21%, Medium 41%, High 39% — so we compute balanced class weights during training rather than assuming an even split. And after filtering to English and combining Subject and Body, we found zero exact duplicate tickets, so deduplication is more of a safety check on this dataset than a real cleanup step."

Slide 6 — Preprocessing Pipeline (A · ~1:00)

"That feeds a six-step pipeline. We take the raw Subject and Body, filter to English only and combine them into one text field, deduplicate before splitting, vectorize with Keras's TextVectorization layer over a 5,726-word vocabulary — which also handles lowercasing and punctuation stripping for us — pad every sequence to a 57-token max length, and encode the three priority labels as integers zero through two. Here's an example trace — a raw Subject-plus-Body ticket gets normalized, then turned into a fixed-length sequence of integers the model can actually read."

Transition: "Now Juan José will take you through the model itself."

Slide 7 — Why Bi-LSTM (JJ · ~1:05)

"Thanks, Andrea. Why a Bidirectional LSTM specifically? We considered a few options. A CNN only sees a fixed local context window. A regular, one-directional LSTM only sees what came before a word, not what comes after. Transformers like BERT would've been a great option, but they're explicitly out of scope for this assignment, which calls for ANN, CNN, or RNN architectures. A Bidirectional LSTM gave us the best fit: it reads the sequence both forward and backward, so at every word it has the full context of the sentence — past and future — at a fraction of the parameters a transformer needs. You can see that here: the word 'twice' in 'payment failed twice' carries a lot more urgency once the model already knows the sentence ends with 'no access.' We run a 128-unit forward LSTM and a 128-unit backward LSTM and concatenate them into a 256-unit representation."

Slide 8 — Model Architecture (JJ · ~1:10)

"Here's the full layer stack. Tokenized text comes in at a fixed shape of 57, goes through a trainable 64-dimensional embedding layer over our 5,726-word vocabulary, then the bidirectional LSTM layer producing 256 hidden units, 30% dropout, and finally a dense softmax layer producing our three priority probabilities. Every hyperparameter here was a deliberate choice: 64-dimensional embeddings balance expressiveness against parameter count for a fairly small vocabulary, 128 units per direction gives enough capacity without being excessive, and the sequence length of 57 was carried over as a fixed cap. We trained with Adam and sparse categorical cross-entropy, using Keras-Tuner to search the hyperparameter space rather than tuning by hand."

Slide 9 — Training Results (JJ · ~1:15)

"So how did it perform? 65% test accuracy, with a Macro F1 of 0.64 — meaning performance is fairly consistent across all three classes, not propped up by one easy class. Breaking it down further: High and Medium both score around 0.65 to 0.67 F1, while Low is a bit weaker at 0.61, since it's our smallest class at only 21% of the data even with balanced class weights applied. That number needs context: random guessing across our three classes gets you 33%, so we're at roughly 1.95 times better than chance, with meaningful signal across all three priority levels. Here's an example: a ticket about a payment failing twice gets classified High with 78% confidence, Medium a distant second at 17%. We think 65% is genuinely sufficient for an MVP, for two reasons. First, this task is inherently ambiguous — human agents disagree with each other on priority all the time, so 33% chance is really the honest floor, not zero. Second, the confidence scores act as a safety net: low-certainty predictions can be routed to a human for review, so a high-priority ticket is never silently auto-dismissed."

Slide 10 — Overfitting Prevention (JJ · ~0:55)

"We took overfitting seriously given a relatively small training set. Three defenses: 30% dropout after the LSTM layer, chosen via the Keras-Tuner search, forcing the network to learn redundant rather than memorized representations. Early stopping, which monitors validation loss and halts training — restoring the best weights — before the model starts memorizing the training set. And a stratified 70/15/15 split so every split has the same class distribution, giving an honest read on generalization. On top of that, deduplication happens before splitting to avoid leakage, and since our classes are imbalanced — Low 21%, Medium 41%, High 39% — we compute balanced class weights rather than letting the model default to the majority classes, and the automated hyperparameter search avoids hand-tuning on the validation set."

Transition: back to Andrea for integration.

Slide 11 — MVP Integration (A · ~1:05)

"Bringing it all together: an agent enters a Subject and Body into the Streamlit UI, which sends an HTTP POST to our FastAPI backend's `/predict` endpoint. The backend loads the saved model and tokenizer once at startup, applies the exact same preprocessing used in training, runs prediction, and returns a JSON response with the priority, the full confidence breakdown, and the routed department — end to end, in under a second. The API contract was fixed on day one specifically so frontend and backend could build in parallel — Dalton's Streamlit app and my FastAPI service never had to wait on each other."

Slide 12 — Live Demo (D · ~0:50)

"And now, let's see it actually working. Four steps: paste in a Subject and Body from a real customer email, click Classify, and within a second you get back the priority, the confidence breakdown, and the recommended department — logged to session history so you can see everything classified so far. *(Switch to live demo; fall back to this simulated output if needed: High, 78% confidence, routed to Technical Support.)*"

Slide 13 — Thank You (D, all wrap up · ~0:15)

"That's our Support Ticket Priority Classifier. Thank you — we're happy to take any questions."

Timing summary

#	Slide	Speaker	Time	Cumulative
1	Title	D	0:25	0:25
2	The Problem	D	1:05	1:30
3	Business Value	D	0:55	2:25
4	Our Solution	A	1:05	3:30
5	Dataset & EDA	A	1:05	4:35
6	Preprocessing	A	1:00	5:35
7	Why Bi-LSTM	JJ	1:05	6:40
8	Model Architecture	JJ	1:10	7:50
9	Training Results	JJ	1:15	9:05
10	Overfitting Prevention	JJ	0:55	10:00
11	MVP Integration	A	1:05	11:05
12	Live Demo	D	0:50	11:55
13	Thank You	D	0:15	12:10

Total speaking time ≈ **12:10**, leaving ~50 seconds of buffer across the deck for slide transitions, the live demo running live (rather than narrated), and natural pauses — landing the full run at the **13-minute** mark.

Speaking-time split: Dalton ~4:20 (intro/problem/value/demo/close) · Andrea ~4:15 (solution/data/preprocessing/integration) · Juan José ~4:25 (model/results/regularisation) — evenly balanced across the three of you.